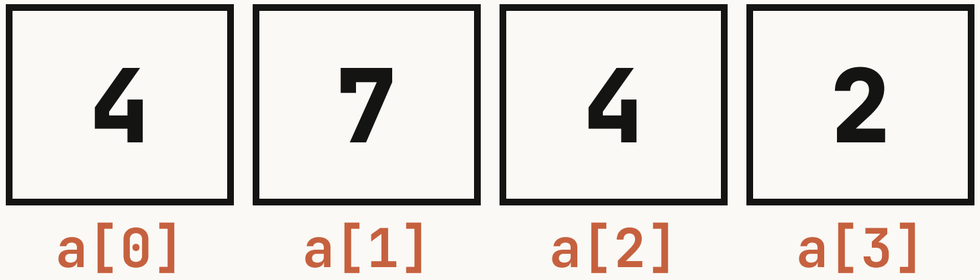
LESSON 13

有没有 == 在哪里

数组遍历与简单查找

学完以后，给你一串数和一个目标，程序能说出它在不在、有几个、排第几。

7 号在不在？在第几个？



一串号码已经存进数组。拿着一个目标数 x，可以问三个问题：

有没有

x 出现过吗？

有几个

x 一共出现几次？

在哪里

x 第一次出现在哪个下标？

办法都一样：从 a[0] 开始，一个一个和 x 比。

有没有：准备一个“找到了吗”的盒子

find.cpp

```
13      bool found = false;
14      for (int i = 0; i < n; i++) {
15          if (a[i] == x) {
16              found = true;
17          }
18      }
```

循环后：if (found) 输出 YES，否则输出 NO。
先读 n 和 n 个数，最后读 x，都是 -1000 到 1000 的整数。

bool 盒子只放两种值：true（是）或 false（不是）。

先当作没找到，碰到一样的，就改成 true。

输入 4，再输入 4 7 4 2，再输入 7

YES

改一改：== 的结果放进

same.cpp

```cpp
#include <iostream>
using namespace std;
int main() {
    int num = 4;
    int x = 7;
    bool same = num == x;
    cout << same << endl;
    return 0;
}
```

第 7 行输出

0

点黄色的数字，改完结果马上变。先猜再改：7 和 7，0 和 0。

cout 输出 bool 时，true 显示 1，false 显示 0，和第 3 课比较的结果一样。

find.cpp

4 7 4 2 里找 7：found 怎么变

a[0] 是 4　4 == 7 不成立	found 还是 false
a[1] 是 7　7 == 7 成立	found 变成 true
a[2] 是 4　4 == 7 不成立	found 还是 true
a[3] 是 2　2 == 7 不成立	还是 true，输出 YES

found 只会从 false 变成 true，没有哪一句把它变回去。

所以只要中间碰到过一次，最后就是 YES。

小测：4 7 4 2 里找 5，输出什么？

1 YES

2 NO

3 NO NO NO NO

4 什么也没有

答对了！found 一直是开头的 false，所以输出 NO。

小测：4 7 4 2 里找 2，输出什么？

1 YES

2 NO

3 3

4 1

答对了！2 在最后一个盒子 a[3]，i 走到 3 时比到了。

动手改一改

从 find 开始。不问有没有，改问有几个：想想第 9 课的计数器。

出现了几次?

先读入 n（1 到 100）和 n 个数，最后读入 x，都是 -1000 到 1000 的整数。输出 x 出现了几次。

4 7 4 2 里找 4

2

4 7 4 2 里找 5

0

Dev-C++：打开 starter/03_count.cpp

猜一猜：找到就记下 i

where.cpp

```
13      int pos = -1;
14      for (int i = 0; i < n; i++) {
15          if (a[i] == x) {
16              pos = i;
17          }
18      }
```

屏幕上

2

这样记下的是**最后一次**出现的位置。

想要第一次的，就要让后面的不能再改它。

只记第一次：pos 还是 -1 才记

first.cpp

```
13      int pos = -1;
14      for (int i = 0; i < n; i++) {
15          if (a[i] == x && pos == -1) {
16              pos = i;
17          }
18      }
```

4 7 4 2 里找 4

0

下标最小是 0，不会是 -1。pos 还是 -1，说明还没找到过。
第一次找到后 pos 不再是 -1，后面的就记不进去了。

小测：first.cpp 在 4 7 4 2 里找 9？

1 0

2 4

3 什么也没有

4 -1

答对了！一次也没找到，pos 还是开头的 -1。-1 不是下标，正好用来表示“没有”。

小测：写成 pos = a[i]，找 7 输出几？

数组 4 7 4 2，找 7

1 7

2 1

3 2

4 -1

答对了！a[i] 是盒子里的数，就是 7 本身。想要位置要写 i。

动手改一改

从 first 开始。这次人说的“第几个”，从 1 数起。

排第几个?

读入同上。输出 x 第一次出现是第几个，从 1 数起；一次也没有就输出 0。

4 7 4 2 里找 4	4 7 4 2 里找 2	4 7 4 2 里找 9
1	4	0

Dev-C++: 打开 starter/05_rank.cpp

读报错：false 拼错了

find.cpp（只阅读）

```
13      bool found = flase;
```

```
13:18: error: 'flase' was not declared in this scope
```

true 和 false 是 C++ 认识的词。拼错了，就成了一个没准备过的盒子名字。

有的编译器还会提示 did you mean 'false'，意思是“你是不是想写 false”。按提示改成 false 就好。

Dev-C++ 显示的文字和位置可能不一样，还是先找行号和 error。

猜一猜：== 少写了一个等号

find.cpp（只阅读）

```
15          if (a[i] = x) {
16              found = true;
17          }
```

屏幕上

YES

放进去的 9 不是 0，if 就当成立，明明没有 9 也说 YES。

能运行、没有 error，答案却错了。**比较一定写两个等号。**

找错：哪一行把 found 改回去了？

find.cpp（只阅读）

```
1       for (int i = 0; i < n; i++) {
2           if (a[i] == x) {
3               found = true;
4           } else {
5               found = false;
6           }
7       }
```

点代码里的一行

这段想回答“有没有”。

4 7 4 2 里找 7，明明有，屏幕上却是 NO。

点出把 found 改回 false 的那一行。

自己挑战

中了几个号

先读入 n 和你选的 n 个号码，再读入 m 和开出的 m 个号码。n、m 是 1 到 100，号码是 1 到 1000，同一组里没有重复。输出你的号码里有几个被开出来了。

输入

```
5
3 8 12 20 33
4
8 9 20 1
```

输出（8 和 20 中了）

```
2
```

先自己写，卡住了再翻下一页。

卡住了？一次看一条提示

提示 1　两组号码都要存下来，准备两个数组。

提示 2　先想一个号码：怎么知道它在不在开出的号码里？这就是今天的“有没有”。

提示 3　你的每个号码都要这样查一遍：外面一层循环，里面再一层。

造一个签到小程序

先存下一组号码，再让别人来查自己在不在。主题你自己定。

先做到

读入 `n` 个号码和一个 `x`，说出 `x` 在不在。

选你喜欢的

班级签到、借书查询、抽奖查号、宝藏地图，都可以。

还想更厉害

用 `while` 一直查，输入 `0` 才停；还能说出是第几个。

没有标准答案。写之前说清想要的结果，写完试试最小和最大的输入。

和同桌换着玩

1

换着运行

运行同桌的程序，查一个有的、一个没有的

2

说一个喜欢的

告诉同桌，你最喜欢这个程序哪里

3

提一个想法

问问同桌：查到以后，还可以告诉我什么？

听完再改一版。想一想、做出来、试一试、分享、再改，程序就是这样越做越好的。

说得清吗?

Q1 **有没有、有几个、在哪里，各用什么盒子记?**

有没有用 `bool found`，有几个用 `count`，在哪里用 `pos`。

Q2 **为什么不能写 `else found = false`?**

后面没对上的数会把它改回去，最后只看最后一个数。

Q3 **`i` 和 `a[i]` 有什么不同?**

`i` 是位置，`a[i]` 是那个位置里放的数。

速查卡

```
for (…) { if (a[i] == x) { … } }
```

`bool found`	true 或 false，有没有
`count++`	有几个
`pos = i`	在哪里；先放 -1 表示还没找到
`pos == -1`	加上它，只记第一次
`==` `=`	两个等号是比较，一个是放进去

这些后面再学

break

找到了就提前结束循环

二维数组

一张表格，用行和列取数

排序后查找

排好队以后，可以找得更快

今天学会查找，一个一个比，再用对的盒子记下你要的答案。

谢谢！

喜欢这节课？欢迎来 GitHub 和我交流。

 cpp-for-youth

下一课：二维数组入门